

Cost Floor & Llama Routing

What runs on Meta's Llama for free, what levers hold live-hosting cost to the floor, and how it's all ON from day one — with a load test that keeps it that way.

Bottom line: From the get-go, the platform's AI runs entirely on **Meta's Llama via Groq's free tier** (projected LLM spend $\approx$ \$0), and every **live-hosting cost lever** is set to its **minimum**. Nothing here is behind a switch you have to remember to flip — it's the default state. Live *video* hosting stays counsel-gated (off until your attorney clears it); these levers just make it as cheap as possible the moment it turns on. A load test (`deploy-kit/load-test.mjs`) asserts this whole posture and fails the build if anything drifts off the floor.

1. AI — everything that can run on Llama does, for free

The platform calls an LLM from $\sim$ 190 places. Every call names a *job* ("routine", "reasoning", "document", ...) and a central router (`backend/sdk/ai-models.ts` + `integrations.ts`) decides which model runs it. The shipped defaults point the whole thing at Groq, which serves Meta's Llama models on its free tier:

- `LLM_PROVIDER = groq` — Meta's Llama, no GPU to run, no per-token bill on the free tier.
- `AI_FORCE_CHEAP_TIER = 1` — the cheap-tier brake is ON, so even calls that ask for a bigger model stay on Llama.
- `PROVIDER_STT = groq` — speech-to-text (Whisper) is on the free tier too.
- Small tier $\rightarrow$ `llama-3.1-8b-instant` ; reasoning/large tier $\rightarrow$ `llama-3.3-70b-versatile` .

On Groq even the "frontier" tier alias resolves to Llama-70B, so **no paid model (OpenAI/Anthropic) is ever hit** unless an admin deliberately switches the provider. That's the "everything that can run on Llama, does" guarantee — Llama the model is free to use, and Groq's free tier means running it costs nothing at your scale. (If you later want a stronger paid model for a specific job — say heavy legal-document drafting — you point just that one job at it with `AI_JOB_MODEL_DOCUMENT` and turn the brake off. Nothing else changes.)

2. Live hosting — every cost lever at the floor

Live video is the one place cost scales, and it scales with **egress bandwidth (concurrent viewer-hours)**, not users. New settings (all live by default, in the "Scale" category) cap that egress at the minimum:

Lever	Floor default	Effect
<code>HOSTING_COST_FLOOR_MODE</code>	ON	Applies all caps below to every session
<code>HOSTING_MAX_BITRATE_KBPS</code>	800	$\sim$ 0.36 GB/viewer-hour — $\sim$47% cheaper than a 1.5 Mbps stream
<code>HOSTING_MAX_RESOLUTION</code>	640 $\times$ 360	360p — legible for screen-share, minimal bandwidth
<code>HOSTING_MAX_FRAMERATE</code>	15	Smooth enough for storefront/app share, half the frames
<code>HOSTING_MAX_VIEWERS_PER_ROOM</code>	50	Caps the worst-case egress a single room can generate
<code>HOSTING_MAX_LIVE_HOURS_PER_DAY</code>	4	Bounds viewer-hours/month at the source
<code>HOSTING_UNLOCK_ENABLED</code>	ON	Earn-to-unlock gate — caps how many people can go live at once
<code>HOSTING_PREFER_VOD</code>	ON	Nudges non-live content to cheap record-and-serve over real-time forwarding

The token endpoint (`sessionLiveKitToken`) returns these caps as `limits` , the **Host Studio constrains its screen-share encode to them** (single low-bitrate layer, capped resolution/fps), and the endpoint **refuses viewers past the per-room cap**. So a session physically cannot exceed the floor.

These caps do not enable hosting. `SESSION_HOSTING_ENABLED` stays OFF (counsel-gated). The floor is pre-set so that the day counsel clears it, it runs cheap.

3. It's on from day one — and a load test keeps it that way

`node deploy-kit/load-test.mjs` reads the real source and asserts three things, failing (exit 1) if any regress:

1. **Defaults** — the code actually ships at the floor (every setting above at its floor value; the counsel gate still off).
2. **AI router** — under those defaults every job resolves to a Llama model, projected spend \$0, and the router sustains millions of resolutions/sec (a load test of the hot path).
3. **Hosting** — the egress math at the floor caps, across the three 200K-user scenarios plus a concurrency surge, stays within the floored band and well under the 1.5 Mbps baseline (Light/Moderate/Heavy all ~14–44% cheaper than baseline).

This sits alongside the existing traffic load test (`deploy-kit/loadtest.k6.js` , `LOAD-TEST-PLAN.md`) — that one measures real per-task capacity; this one guards the cost posture. An admin can read the live state any time via the `costFloorStatus` endpoint.

4. Live-hosting autoscaling — scales on viewers, to zero when idle

The app already autoscales its web replicas by request rate (`infraScaleController`). Live hosting is a **separate media tier** (the LiveKit SFU + TURN) whose cost is driven by **concurrent viewers**, not API traffic — so it gets its own autoscaler, `livekitScaleController` , running every minute:

- **Load signal = concurrent viewers**, summed from active LiveKit sessions. Desired nodes = `ceil(viewers ÷ viewers-per-node)` ($\approx 1,000$ viewers/node), clamped to `[min, max]` , stepped per tick so it never thunders.
- **Scales to zero.** `LIVEKIT_SCALE_MIN_NODES = 0` , so when nobody is live the media tier runs **0 nodes = \$0**. Because hosting is counsel-gated off, there are no viewers yet — it simply holds at zero until hosting is cleared and someone goes live. The instant real viewers arrive it spins up; when they leave it returns to zero.
- **Scales up under load, with a burst ceiling.** It grows node-by-node as viewers climb, up to an emergency ceiling (`LIVEKIT_SCALE_MAX_NODES = 20` $\approx 20,000$ concurrent viewers), and can auto-burst past an optional dollar budget only for the minutes a surge actually lasts.
- **ON from day one, safe until credentialed.** `LIVEKIT_SCALE_ENABLED = 1` , provider `none` (decide-only). When you self-host LiveKit on Hetzner/OVH/Vultr/AWS, point `LIVEKIT_SCALE_PROVIDER` at a small node-pool webhook (or a Railway LiveKit service) and it starts acting. Same guard/budget discipline as the app scaler — never below min, never above the ceiling, bounded per tick.

The load test asserts the scale-to-zero-at-idle and ceiling-clamp behavior, so this posture can't silently drift either. Read the live state (viewers, nodes, cost) any time by calling `livekitScaleController` with `{ "dry_run": true }`.

Everything above is the day-one default. The only cost that ever grows is live-video egress, and only after counsel clears hosting — and even then it's held to the floor by the caps here and sized by an autoscaler that returns to zero when idle. See `LIVEKIT-HOSTING-COST-MODEL.md` for the full unit economics and the plug-in calculator.